

5주 차 보고서

(2025년 1 월 25 일 ~ 2026년 1 월 31 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

프레임 워크에 Lock과 DeadLockProfiler를 추가했다. 멀티쓰레드 환경에서 데이터의 안정성을 보장하기 위해 사용하는 RW SpinLock을 사용하여 구현해보았다. 동시 읽기는 허용하되, 쓰기는 독점적으로 제어하여 데이터 일관성을 지키는 장치를 만들었다.

락을 추가했으니 이 락들이 서로 꼬여서 발생하는 데드락 문제가 빈번하게 발생할 수 있는데 이때 데드락을 사전에 감지하고 예방하기 위해 DeadLockProfiler를 만들었다. PushLock은 락을 획득할 때 호출하며 현재 잡고 있는 락 -> 지금 잡으려는 락의 관계를 저장하고 처음 보는 순서라면 사이클 검사를 요청한다. PopLock은 락을 해제할 때 호출한다. 현재 잡고 있는 락 목록에서 제거하여 순서 추적을 유지한다. CheckCycle은 그래프에 순환이 있는지 검사를 하고 Dfs는 그래프를 파고들며 순환 구조가 있는지 찾아내는 알고리즘이다.

메모리에 관한 내용을 추가로 공부했다.

```

/*-----
:   RefCountable
-----*/

class RefCountable
{
public:
    RefCountable() : _refCount(1) {}
    virtual ~RefCountable() {}

    int32 GetRefCount() { return _refCount; }

    int32 AddRef() { return ++_refCount; }
    int32 ReleaseRef()
    {
        int32 refCount = --_refCount;
        if (_refCount == 0)
        {
            delete this;
        }
        return refCount;
    }

protected:
    atomic<int32> _refCount;
};

```

New로 생성한 객체를 적절한 시점에 delete하지 않으면 메모리 누수가 발생하는데 RefCountable을 만들어서 사용하는 곳이 아무곳도 없을 때 스스로 파괴하도록 자동화 시스템을 만들었다.

```

template<typename Type, typename... Args>
Type* xnew(Args&&... args)
{
    Type* memory = static_cast<Type*>(xalloc(sizeof(Type)));
    new(memory)Type(forward<Args>(args)...); // placement new
    return memory;
}

template<typename Type>
void xdelete(Type* obj)
{
    obj->~Type();
    xrelease(obj);
}

```

```

#ifdef _DEBUG
#define xalloc(size)      BaseAllocator::Alloc(size)
#define xrelease(ptr)    BaseAllocator::Release(ptr)
#else
#define xalloc(size)      BaseAllocator::Alloc(size)
#define xrelease(ptr)    BaseAllocator::Release(ptr)
#endif

```

C++의 기본연산자인 `new` 대신에 `xnew`를 만들어서 수동으로 정밀하게 제어하는 코드를 만들었다. `xalloc`으로 순수 메모리 공간을 확보하고, 해당 위치에 객체 생성자를 강제로 호출한 후, 완성된 객체 포인터를 반환한다. `xdelete`는 객체의 소멸자를 명시적으로 호출하여 자원을 정리하고, `xrelease`를 통해 실제 메모리 공간을 해제한다.

```

class BaseAllocator
{
public:
    static void* Alloc(int32 size);
    static void Release(void* ptr);
};

```

```

void* BaseAllocator::Alloc(int32 size)
{
    return ::malloc(size);
}

void BaseAllocator::Release(void* ptr)
{
    ::free(ptr);
}

```

```

template<typename T>
class TSharedPtr
{
public:
    TSharedPtr() {}
    TSharedPtr(T* ptr) { Set(ptr); }

    // 복사
    TSharedPtr(const TSharedPtr& rhs) { Set(rhs._ptr); }
    // 이동
    TSharedPtr(TSharedPtr&& rhs) { _ptr = rhs._ptr; rhs._ptr = nullptr; }
    // 상속 관계 복사
    template<typename U>
    TSharedPtr(const TSharedPtr<U>& rhs) { Set(static_cast<T*>(rhs._ptr)); }

    ~TSharedPtr() { Release(); }

public:
    // 복사 연산자
    TSharedPtr& operator=(const TSharedPtr& rhs)
    {
        if (_ptr != rhs._ptr) {
            Release();
            Set(rhs._ptr);
        }
        return *this;
    }
}

```

```

// 이동 연산자
TSharedPtr& operator=(TSharedPtr&& rhs)
{
    Release();
    _ptr = rhs._ptr;
    rhs._ptr = nullptr;
    return *this;
}

bool operator==(const TSharedPtr& rhs) const { return _ptr == rhs._ptr; }
bool operator==(T* ptr) const { return _ptr == ptr; }
bool operator!=(const TSharedPtr& rhs) const { return _ptr != rhs._ptr; }
bool operator!=(T* ptr) const { return _ptr != ptr; }
bool operator<(const TSharedPtr& rhs) const { return _ptr < rhs._ptr; }
T* operator*() { return _ptr; }
const T* operator*() const { return _ptr; }
operator T* () const { return _ptr; }
T* operator->() { return _ptr; }
const T* operator->() const { return _ptr; }

bool IsNull() { return _ptr == nullptr; }

```

```

private:
    inline void Set(T* ptr)
    {
        _ptr = ptr;
        if (ptr)
            ptr->AddRef();
    }

    inline void Release()
    {
        if (_ptr != nullptr)
        {
            _ptr->ReleaseRef();
            _ptr = nullptr;
        }
    }

private:
    T* _ptr = nullptr;
};

```

```

class Wraight : public RefCountable
{
public:
    int _hp = 150;
    int _posX = 0;
    int _posY = 0;
};

using WraightRef = TSharedPtr<Wraight>;

class Missile : public RefCountable
{
public:
    void SetTarget(WraightRef target)
    {
        _target = target;
        // 중간에 개입 가능
        //target->AddRef();
    }

    bool Update()
    {
        if (_target == nullptr)
            return true;

        int posX = _target->_posX;
        int posY = _target->_posY;

        // TODO : 쫓아간다

        if (_target->_hp == 0)
        {
            _target->ReleaseRef();
            _target = nullptr;
            return true;
        }

        return false;
    }

    WraightRef _target = nullptr;
};

using MissileRef = TSharedPtr<Missile>;

```

```

int main()
{
    WraightRef wraight(new Wraight());
    wraight->ReleaseRef();
    MissileRef missile(new Missile());
    missile->ReleaseRef();

    missile->SetTarget(wraight);

    // 레이스가 피격 당함
    wraight->_hp = 0;
    //wraight->ReleaseRef();
    //delete wraight;
    wraight = nullptr;

    while (true)
    {
        if (missile)
        {
            if (missile->Update())
            {
                missile->ReleaseRef();
                missile = nullptr;
            }
        }

        //missile->ReleaseRef();
        //delete missile;
        missile = nullptr;
    }
}

```

앞서 만든 RefCountable 시스템을 실제로 사용하기 위한 스마트 포인터 구현체와 활용한 코드이다.
C++의 shared_ptr과 유사하게 TSharedPtr<T>를 만들었다.


```

/*-----
| STLAllocator
|-----*/

template<typename T>
class StlAllocator
{
public:
    using value_type = T;

    StlAllocator() {}

    template<typename Other>
    StlAllocator(const StlAllocator<Other>&) {}

    T* allocate(size_t count)
    {
        const int32 size = static_cast<int32>(count * sizeof(T));
        return static_cast<T*>(Xalloc(size));
    }

    void deallocate(T* ptr, size_t count)
    {
        xrelease(ptr);
    }
};

```

```

template<typename Type>
using Vector = vector<Type, StlAllocator<Type>>;

template<typename Type>
using List = list<Type, StlAllocator<Type>>;

template<typename Key, typename Type, typename Pred = less<Key>>
using Map = map<Key, Type, Pred, StlAllocator<pair<const Key, Type>>>;

template<typename Key, typename Pred = less<Key>>
using Set = set<Key, Pred, StlAllocator<Key>>;

template<typename Type>
using Deque = deque<Type, StlAllocator<Type>>;

template<typename Type, typename Container = Deque<Type>>
using Queue = queue<Type, Container>;

template<typename Type, typename Container = Deque<Type>>
using Stack = stack<Type, Container>;

template<typename Type, typename Container = Vector<Type>, typename Pred = less<typename Container::value_type>>
using PriorityQueue = priority_queue<Type, Container, Pred>;

using String = basic_string<char, char_traits<char>, StlAllocator<char>>;
using WString = basic_string<wchar_t, char_traits<wchar_t>, StlAllocator<wchar_t>>;

template<typename Key, typename Type, typename Hasher = hash<Key>, typename KeyEq = equal_to<Key>>
using HashMap = unordered_map<Key, Type, Hasher, KeyEq, StlAllocator<pair<const Key, Type>>>;

template<typename Key, typename Type, typename Hasher = hash<Key>, typename KeyEq = equal_to<Key>>
using HashSet = unordered_set<Key, Hasher, KeyEq, StlAllocator<Key>>;

```

C++ 표준 라이브러리인 STL 컨테이너가 `xalloc`, `xrelease`를 사용하도록 연결해주는 어댑터 역할을 하는 클래스이다. `StlAllocator`의 핵심 역할은 STL이 메모리가 필요할때마다 표준 할당자 대신 `xalloc`을 호출하게끔 통로를 변경해준다.

```
class StompAllocator
{
    enum {PAGE_SIZE = 0x1000 };
public:
    static void* Alloc(int32 size);
    static void Release(void* ptr);
};

void* StompAllocator::Alloc(int32 size)
{
    const int64 pageCount = (size + PAGE_SIZE - 1) / PAGE_SIZE;
    const int64 dataOffset = pageCount * PAGE_SIZE - size;
    void* baseAddress = ::VirtualAlloc(NULL, pageCount * PAGE_SIZE, MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    return static_cast<void*>(static_cast<int8*>(baseAddress) + dataOffset);
}

void StompAllocator::Release(void* ptr)
{
    const int64 address = reinterpret_cast<int64*>(ptr);
    const int64 baseAddress = address - (address % PAGE_SIZE);
    ::VirtualFree(reinterpret_cast<void*>(baseAddress), 0, MEM_RELEASE);
}
```

`StompAllocator`는 메모리 오염 및 오버런을 즉각적으로 잡아내는 디버깅에 특화되어있다. OS의 가상 메모리를 직접 컨트롤 해서 메모리 범위가 조금이라도 벗어나서 읽거나 쓰는 순간 프로그램이 크래쉬나게 한다.

다음 주 할 일

메모리에 대한 공부를 많이 했는데 여태까지 공부한 것들은 성능 면에서는 좋지 않았다. OS 커널까지 가야하기 때문이다. 다음 주에는 성능을 극대화하기 위해 많이 사용하는 메모리 풀에 대해서 공부할 예정이다.

특이사항